

Code Crafting no. 4

**The Language Stack:
From Silicon to Semantics**

**Set Lonnert
with AI Coöperation**

Foreword

Every few years a book appears that teaches programmers how programming languages work. The subject has attracted good writers. *Crafting Interpreters* [Nystrom, 2021] is scrupulously written and freely available; *Engineering a Compiler* [Cooper and Torczon, 2022] is thorough and technically careful; Andrew Appel’s *Modern Compiler Implementation* [Appel, 1998] teaches by building a complete language called Tiger. The Dragon Book—Aho, Lam, Sethi, and Ullman’s *Compilers: Principles, Techniques, and Tools* [Aho et al., 2006]—sat on departmental reading lists for decades and in revised form still does.

This book does not replace any of them. It exists because none of them quite addresses the question this book is trying to answer.

The question. The question is not *how do I build a compiler?* It is *what kind of thinking does a language embody, and how does that thinking propagate from hardware to syntax to type system?* The difference matters because language design is not a solved engineering problem. It is, in part, a philosophical one. What kinds of things should a type system be able to say? When is a constraint a safety property and when is it a limitation? Why does ownership change what you can express, and what is it that ownership is owning?

These questions do not have algorithmic answers. But they do have precise ones, and finding them requires the kind of reasoning that compilers reward: close attention to definitions, the ability to hold a hierarchy of abstractions in mind simultaneously, and comfort with the idea that a choice made at the type-theory level has consequences that propagate down to the register allocator.

The books that exist. The Dragon Book is comprehensive. Its chapters on data-flow analysis and type checking are authoritative. It is also, by design, a reference text—dense, organized for lookup, concerned with the kind of production compiler that handles FORTRAN or Ada. A reader working through it for the first time is trying to understand a cathedral by studying the maintenance manual. That is one way to learn architecture, but it is not the only way, and it is not the fastest.

Appel’s *Modern Compiler Implementation* has the right instinct: one example language (Tiger), built from the first chapter. The book is genuinely educational. It is also, at this point, a quarter century old, and the ML/C/Java editions feel like three incomplete versions of the same idea. The type system it builds is not the type system interesting languages now have.

Crafting Interpreters is, on its own terms, exceptional. Nystrom writes with unusual clarity, and the two-implementation strategy—tree-walking interpreter first, bytecode VM second—is pedagogically sound. The book stops, by design, before the territory becomes formally interesting. There are no types worth calling a type system, no ownership semantics, no abstract machines in the sense that connects to operational semantics. The language that emerges is a pleasant scripting language. That is not a criticism—it is the intent.

Engineering a Compiler is the most accurate of the four, and the most organized. It is also the hardest to use as a first encounter. It is organized the way a compiler is organized, not the way understanding develops.

What this book does differently. Four things, taken seriously.

Hardware is not abstracted away. Real instruction sets—the 6502 as a historical lens, RISC-V as the modern target—appear as working systems with costs you can measure. Register allocation means something different when you have thirty-two registers than when you have three. The 6502 is not nostalgia. It is a clarity instrument: constrained hardware makes invisible costs visible. At the end of this book, Lark programs run as native machine code on a Raspberry Pi Pico 2W. The target is real.

Theory is earned, not assumed. The type-theory content—Hindley-Milner inference, affine types, trait bounds—appears when the implementation needs it. The formal treatment comes after the reader has seen what the implementation does without it. This ordering is not a simplification; it is the correct epistemological sequence.

One language, start to finish. Every chapter advances Lark—Lambda Affine Resource Kernel—a small, purely functional language with Hindley-Milner type inference, affine ownership, and RISC-V as its compilation target. The reader finishes the book with something working, not a collection of illustrative fragments.

The safety argument is structural. Lark takes Rust’s central insight seriously: memory safety and ownership can be enforced by the type system rather than by a garbage collector. This is not decoration. Affine types are introduced in Chapter 5 and shape every subsequent decision, through the code generator in Chapter 9 and into the formal treatment in Chapter 11. The connection to Rust is named and precise—Lark is not Rust, and the differences matter—as is the connection to Clean, Cogent, and the lineage of functional languages that have treated ownership as a type-theoretic property rather than a runtime mechanism.

The research precedent is Alms [Tov and Pucella, 2011]: OCaml extended with affine types. Lark is approximately Alms, made compilable, with traits instead of OCaml’s module system for structured polymorphism.

The LLM question. There is a practical objection to a book like this in 2026. Large language models can generate parsers, write type checkers, and produce bytecode compilers from a description of the language. A programmer who needs a scanner can ask for one and receive something workable in seconds. Why learn how it works?

The objection proves too much. A surgeon who understands anatomy reasons differently from one who does not, even if a robot makes the incision. A structural engineer who understands load paths thinks differently about a building, even if the stress calculations are automated. The question is not whether the tool exists. It is what kind of understanding the practitioner brings to using it.

Language implementation is where system-level reasoning lives in its most concentrated form. A type system is a formal claim about what programs mean. A register allocator is a constraint-satisfaction problem with a cost model attached. A calling convention is a contract between independently compiled modules. None of these can be outsourced to a tool without the practitioner understanding what the tool is doing. If you do not, you have not outsourced the task—you have abandoned the position from which you could verify the answer.

There is also the specific nature of what language models do with this material. They produce working code from patterns they have seen. They are much weaker at the reasoning that applies when the pattern does not fit: when a design decision in the type system creates a downstream constraint in the code generator, or when a calling convention assumption embedded in an intermediate representation turns out to be wrong for the target architecture. That kind of propagative reasoning—understanding a hierarchy of decisions and their consequences—is exactly what building a language cultivates. It is not separable from the subject.

Lark, in this book, is built with the assistance of large language models. The colophon acknowledges this. The reasoning behind each design decision is in the book and in the repository, not hidden inside a generation process. That combination—tool-assisted construction, human-owned understanding—is the intended model for what technical work looks like in this moment.

Who this book is for. Technically literate programmers who want to understand what is under their language. Not a first programming book—it assumes familiarity with Python or C, preferably both, and with basic data structures. Appropriate for advanced undergraduates, graduate students, and practitioners who have always meant to understand how compilers work and have not found the right entry point.

The book ends at a defined place on the type-theory ladder. Lark sits at Hindley-Milner inference plus affine types plus trait-bounded polymorphism. The next rung is dependent types, where the type checker and the proof checker become the same thing. The Conclusion names that rung and the systems that inhabit it—Idris, Agda, Lean, and the `proof/` kernel in this repository’s own build infrastructure. Readers who want to continue from there have a precise place to go.

Set Lonnert, 2026